How Matrix Size Affects the Speed and Memory of Basic Matrix Multiplication in Python, Java and C

Pablo Castillo Garcerá
Big Data · Individual Assignment 1 · Grado en Ciencia e Ingeniería de Datos · ULPGC · 2026-2027
Code and data: https://github.com/Pablo5250/matrix-multiplication (folder assignment1/)

Abstract

We wrote the same basic matrix multiplication (three nested loops, n^3 multiply-adds) in Python, Java and C, and measured time and memory for square matrices from n = 64 to n = 2048 on one computer (AMD Ryzen 9 5900X, Windows 11). All three programs read the same input files, and their results are identical bit by bit. Python was the slowest by far: 18 to 190 times slower than C, and its largest feasible size was n = 1024 (99 s per multiplication). The time grew close to n^3 in all languages. A surprising result was that Java was faster than C at the tested sizes, which are all powers of two. A small extra experiment showed the reason: at n = 512 and n = 1024 the C program is about 6-7 times slower than at n - 1 or n + 1, because the column accesses all fall into the same few places of the CPU cache. At sizes that are not powers of two, C was the fastest. These conclusions only apply to these programs, settings and this machine.

1. Introduction and objective

Matrix multiplication is a core operation in data science (for example in linear regression and neural networks). Its simplest algorithm does n^3 multiply-adds, so it becomes slow very quickly when n grows. In this first study we want to understand how much the programming language and its runtime change the cost of the same algorithm.

Research question. How does the matrix size n affect the runtime and the memory use of the same basic triple-loop matrix multiplication written in Python, Java and C, on one machine?

Factor we change: the matrix size n (and the language). **What we measure:** time of one multiplication and peak memory of the process.

Hypotheses (written before measuring):

- H1: in all three languages the time grows about like n^3 (doubling n multiplies the time by about 8).
- H2: Python is much slower (around 50-100 times) than C and Java, because it is interpreted.
- H3: C is the fastest and Java is close to C, because both run machine code.
- H4: C uses about the memory of the three matrices ($3 \cdot 8n^2$ bytes); Java and Python use more, because of the JVM and of Python objects.

2. Background

The algorithm. For $C = A \cdot B$, each element is $c[i][j] = \Sigma k\ a[i][k] \cdot b[k][j]$. The basic algorithm uses three loops (i, j, k), so it does n^3 multiplications and n^3 additions: its time complexity is $O(n^3)$ [1]. The memory needed for the data is three n × n matrices; with 64-bit numbers that is $3 \cdot 8n^2$ bytes.

How each language runs the code. C is compiled to machine code before running. Java is compiled to bytecode, and the JVM's JIT (just-in-time) compiler turns the hot methods into machine code while the program runs, so the first runs are slower (that is why we use warm-up runs) [4]. CPython is an interpreter: it runs each bytecode instruction one by one, and every number is a separate object in memory.

The cache. The CPU keeps recently used memory in small, fast caches (L1, L2, L3). Memory is moved in blocks of 64 bytes called cache lines. A cache is divided into sets, and each memory address can only go to one set, which holds a few lines (8 on our CPU's L1 and L2 [9]). If many addresses we use map to the same set, they keep removing each other even when the cache is not full. This is called a conflict miss, and it often happens when the distance between accesses is a power of two [2], [3].

Table 1. Terms used in this report.

Term	Meaning in this report
n	side of the square matrices (n × n)
Median	middle value of the 5 measured runs; one slow run does not move it
IQR	interquartile range = Q3 - Q1, the spread of the middle half of the runs
Warm-up run	a run done before measuring and not counted (lets the JIT and caches settle)
ns / iteration	median time / n^3: the average cost of one c[i][j] += a[i][k]·b[k][j]
Peak memory	largest amount of RAM the whole process used (working set on Windows)
Exponent b	slope of the log-log fit time ≈ a · n^b; b = 3 means cubic growth

3. Methodology

3.1 Implementations

The three programs use the same loop order (i-j-k), the same numeric type (64-bit IEEE-754 double) and the same algorithm. No optimised library is used in the compared code. The only unavoidable difference is how each language stores a matrix:

Table 2. The three implementations. Code: python/matmul.py, java/src/MatMul.java, c/matmul.c.

	Python	Java	C
Storage	list of lists of float objects	double[][] (one array object per row)	one contiguous block of n·n doubles, row-major
Runs on	CPython 3.14.3 interpreter	OpenJDK 25.0.1 (JIT)	gcc 16.2 -O2 -std=c11
Timer	time.perf_counter_ns()	System.nanoTime()	QueryPerformanceCounter
Warm-up runs	1	3	1

3.2 Inputs

One Python script (data/generate_inputs.py) creates A and B for every size with NumPy's PCG64 generator, seed = 42 + n, uniform values in [0, 1). The matrices are saved as raw binary files (float64, little-endian, row-major). All three programs read these same files, so they multiply exactly the same numbers (random generators of different languages give different numbers). data/manifest.json stores the SHA-256 hash of every file.

3.3 Environment

Table 3. Hardware and software (from results/environment.json).

Item	Value
CPU	AMD Ryzen 9 5900X 12-Core Processor
Cores	12 physical, 24 logical
RAM	34.3 GB
Operating system	Windows 11 (10.0.26300)
Python	CPython 3.14.3 (NumPy 2.4.3, psutil 7.2.2, matplotlib 3.10.9)
Java	OpenJDK 64-Bit Server VM (build 25.0.1+8-27, mixed mode, sharing); options -Xmx4g
C compiler	gcc (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r1) 16.2.0; flags -O2 -std=c11
Threads	kernels single-threaded; no BLAS; the JVM also runs its own JIT and GC threads

All runs were done on the same computer, plugged in, with other heavy programs closed. The kernels are single-threaded.

3.4 Correctness checks (before measuring)

- Unit tests in each language: known 2×2 and 3×3 products, $A \cdot I = A$, $I \cdot A = A$, $A \cdot 0 = 0$ and a 1×1 case.
- Comparison with a trusted reference, NumPy A @ B, for n = 16, 64 and 128, using relative tolerance 10^{-9} and absolute tolerance 10^{-12}. With positive inputs the expected rounding error is about $n \cdot 1.1 \cdot 10^{-16}$ (below 10^{-13} here), so 10^{-9} leaves a safe margin and still catches any real bug. NumPy is only the reference, not a compared implementation.
- In every benchmark run the program computes the sum of all elements of C (checksum), which is saved and checked against NumPy. This also makes sure the result is really used.

3.5 Measurement protocol

- **What the timer includes:** only the call multiply(A, B), including creating the output matrix C. Reading the input files is outside the timed part.
- **Repetitions:** each (language, n) runs in a new process: warm-up runs (not counted) and then 5 measured runs. We report the median and the IQR. All raw times are kept in results/raw/times.csv; no value was removed.
- **Sizes:** n = 64, 128, 256, 512, 1024, 2048 (each one doubles the previous).
- **Budgets (decided before running):** 1200 s per configuration and 4096 MB of memory. Before each size, the runner predicts the time from the previous size assuming n^3 growth; if the prediction is over the budget, the size is not started and is reported as skipped.
- **Memory:** the runner reads the peak working set of the process from Windows every 10 ms (psutil [7]). This is the memory of the whole process (runtime + matrices), so we compare it with the size of the three matrices, $3 \cdot 8n^2$ bytes.

3.6 Extra experiment: power-of-two sizes

After seeing that Java was faster than C, we added a small separate experiment (scripts/extra_power_of_two.py): C and Java at n - 1, n and n + 1 for n = 256, 512 and 1024, with the same protocol. It is not part of the main comparison; it only tests one explanation.

4. Results

4.1 Correctness

Table 4. Comparison with NumPy (results/validation.csv).

n	Max abs. error	Max rel. error	All pass	C = Java = Python bit by bit
16	1.8e-15	3.9e-16	yes	yes
64	7.1e-15	4.4e-16	yes	yes
128	2.1e-14	6.3e-16	yes	yes

All unit tests passed, all results are inside the tolerance, and the three languages produce exactly the same bits, as expected, because they add the numbers in the same order. Every benchmark checksum also matched NumPy.

4.2 Runtime

Table 5. Time of one multiplication: median and IQR of 5 measured runs. "× C" = median / median of C at the same n. Peak memory = peak working set of the process.

Language	n	Runs	Median time	IQR	ns / iteration	× C	Peak memory (MiB)
C	64	5	0.112 ms	0.002 ms	0.43	1.00	1.9
C	128	5	1.953 ms	0.023 ms	0.93	1.00	4.9
C	256	5	24.8 ms	0.494 ms	1.48	1.00	6.0
C	512	5	591.6 ms	1.316 ms	4.41	1.00	10.4
C	1024	5	4.90 s	13.8 ms	4.56	1.00	28.4
C	2048	5	40.48 s	222.5 ms	4.71	1.00	100.4
Java	64	5	0.232 ms	0.029 ms	0.89	2.07	45.6
Java	128	5	1.246 ms	0.018 ms	0.59	0.64	46.9
Java	256	5	11.8 ms	0.098 ms	0.70	0.48	51.1
Java	512	5	139.3 ms	0.239 ms	1.04	0.24	69.6
Java	1024	5	1.73 s	142.9 ms	1.62	0.35	138.1
Java	2048	5	33.45 s	441.9 ms	3.89	0.83	361.5
Python	64	5	21.1 ms	2.032 ms	80.46	187.66	14.7
Python	128	5	153.7 ms	15.7 ms	73.27	78.68	16.3
Python	256	5	1.20 s	98.6 ms	71.37	48.20	22.9
Python	512	5	10.87 s	449.0 ms	80.99	18.38	44.9
Python	1024	5	98.61 s	1.85 s	91.84	20.14	135.5

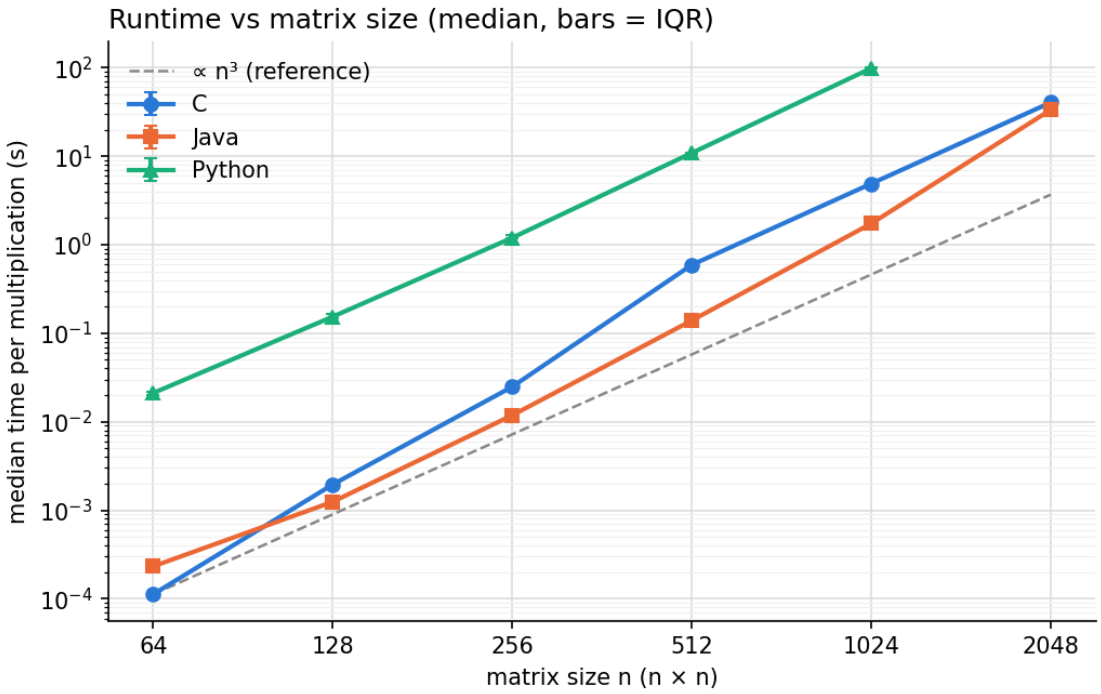


Figure 1. Median time vs n (both axes logarithmic). The dashed line grows like n^3. Error bars (IQR) are very small.

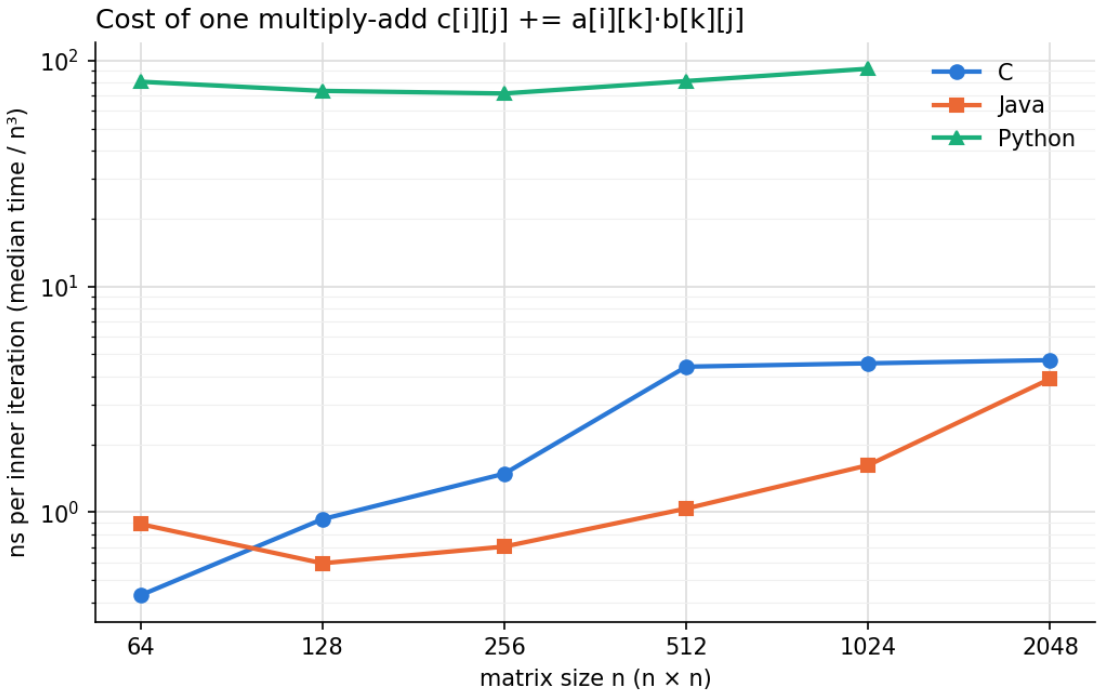


Figure 2. Time per inner iteration (median / n^3). A flat line means perfect n^3 growth.

Table 6. Fitted exponent b for n ≥ 256 (results/slopes.csv).

Language	Sizes used for the fit	Exponent b (time ≈ a·n^b)
C	256 512 1024 2048	3.506
Java	256 512 1024 2048	3.804
Python	256 512 1024	3.182

Table 7. Configurations that were not run.

Language	n	Status	Predicted total time
Python	2048	skipped predicted over budget	4733 s

Limits. The largest tested size was n = 2048 for C and Java and n = 1024 for Python. Python at n = 2048 was not started, because the predicted time (about 4,700 s) was over the 1200 s budget. There were no timeouts or memory failures.

4.3 Power-of-two sizes

Table 8. ns per inner iteration at n - 1, n and n + 1 (results/extra_power_of_two.csv).

Language	n	n - 1	n (power of 2)	n + 1
C	256	0.59	1.32	0.61
C	512	0.64	4.64	0.75
C	1024	0.70	4.38	0.70
Java	256	0.93	0.81	0.70
Java	512	1.11	1.19	1.03
Java	1024	1.88	1.79	1.50

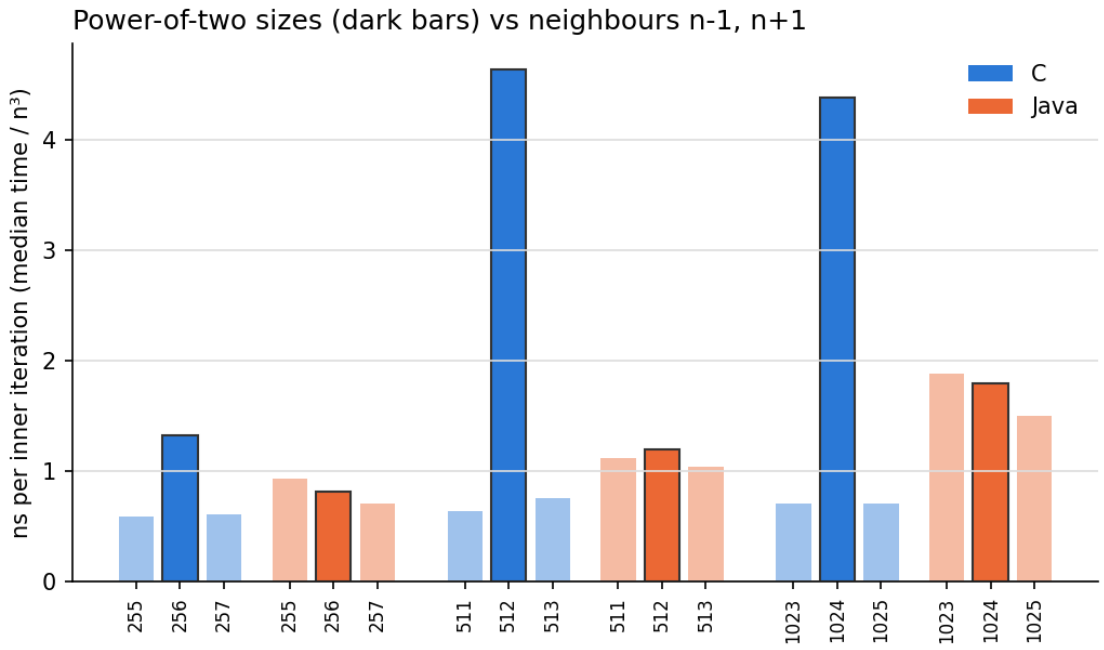


Figure 3. Dark bars: power-of-two sizes. Light bars: the neighbours n - 1 and n + 1.

4.4 Memory

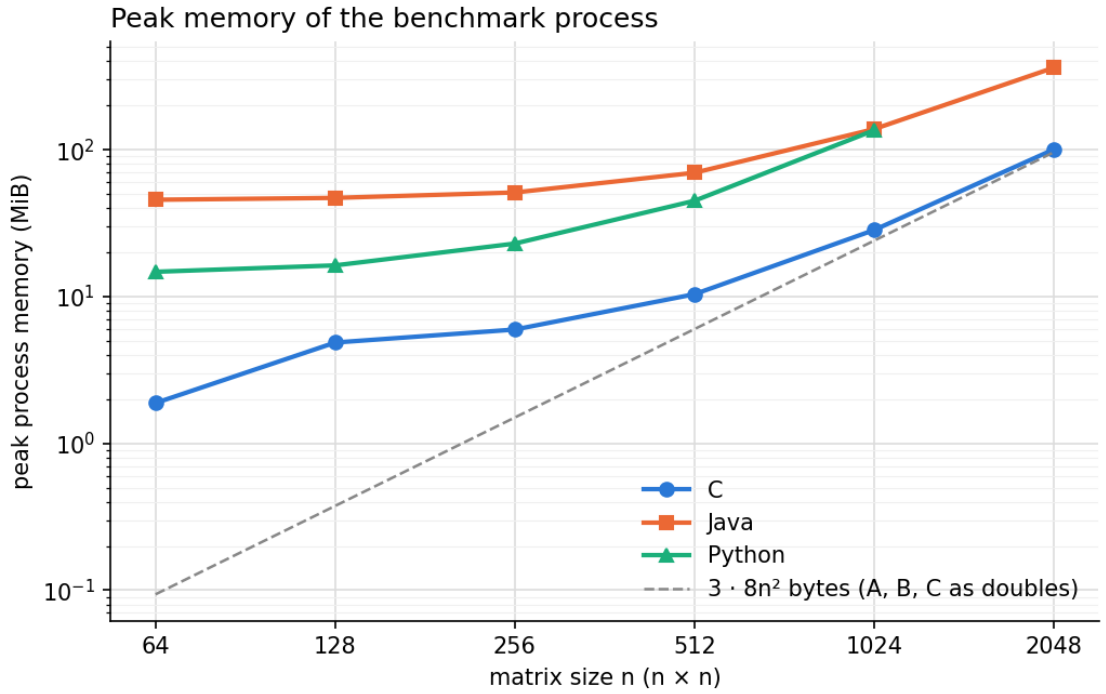


Figure 4. Peak memory of the benchmark process vs n. The dashed line is the size of A, B and C as doubles.

5. Discussion

5.1 Growth with n (H1)

H1 is supported. Python's time per iteration stays between 71 and 92 ns, so its growth is almost exactly cubic (b = 3.05 from n = 64 to 1024). For C, the fitted b = 3.51 is larger, but this comes from a single jump between n = 256 and n = 512 (Figure 2); from n = 512 to 2048 the C time grows with b = 3.05. Java's cost per iteration grows from 0.7 ns to 3.9 ns at n = 2048, so its fitted b = 3.80 is above 3. The algorithm is $O(n^3)$, but the constant (the cost of one iteration) is not fixed: it depends on how well the data fits in the cache.

5.2 Python vs C and Java (H2)

H2 is supported. Python needed 71-92 ns for each multiply-add, while C and Java needed about 0.6-4.7 ns. At n = 1024 Python was 20 times slower than C and 57 times slower than Java; at n = 1023, where C does not suffer from the cache problem, C needs 0.70 ns, about 130 times less than Python. The reason is the interpreter: for every c[i][j] += a[i][k]·b[k][j] CPython runs several bytecode instructions, looks up list elements, reads float objects through pointers and creates a new float object for the result. C and Java do the same work with a few machine instructions.

5.3 Why Java was faster than C (H3)

H3 is rejected for the main sizes: Java was faster than C at every n ≥ 128. The extra experiment explains this. In C, the matrix B is one block, so b[k][j] and b[k+1][j] are exactly 8n bytes apart: 4096 bytes when n = 512. On our CPU, addresses that are 4096 bytes apart go to the same L1 set, which only holds 8 lines [9]. The inner loop reads a full column of B, so almost all of its 512 lines compete for 8 places and have to be read again from slower memory every time. The L2 cache has the same problem at a smaller scale. At n = 511 or 513 the distance is not a power of two, the column is spread over many sets, and C becomes 6-7 times faster (0.64-0.75 ns instead of 4.64 ns). In Java, every row is a separate object with a small header, so the distance between rows is not exactly a power of two and the problem mostly disappears. At sizes that are not powers of two, C was faster than Java (Table 8).

We also checked the machine code that gcc produced (gcc -O2 -S): the inner loop keeps c[i][j] in a register and only does a load, a multiply and an add, so the slowdown does not come from extra instructions in C.

5.4 Memory (H4)

H4 is supported. For C, the peak memory is close to the three matrices at large n (100.4 MiB vs 96 MiB at n = 2048). Java starts at about 45 MiB with tiny matrices, which is the JVM itself, and reaches 361.5 MiB at n = 2048, because each run creates a new result matrix and the garbage collector frees old ones only from time to time. Python used 135.5 MiB at n = 1024, more than 5 times the 24 MiB of data: each number is a float object (24 bytes) plus a pointer in the list (8 bytes), so 32 bytes instead of 8.

5.5 Limitations

- One machine, one operating system and one setting per language; other CPUs have other cache sizes.
- Only 5 measured runs per configuration. The IQR was small (below 3% of the median in 11 of 17 configurations, at most 12%), so the conclusions do not depend on one run.
- Memory is sampled every 10 ms. For very short runs (C with n ≤ 128) the process can end between two samples, so those values may be too low. Peak memory of different runtimes includes very different things (JVM, interpreter), so it is not only the cost of the matrices.
- Windows can run background tasks during measurements; we closed heavy programs but could not control everything.
- We did not test why Java becomes slower at n = 2048 (3.9 ns per iteration). Possible reasons are partial cache conflicts or TLB misses (the column touches 2048 different memory pages).

6. Conclusions

On this machine and with these settings: (1) the basic algorithm grows close to n^3 in the three languages, but the cost per iteration changes with the size because of the cache; (2) Python is 18-190 times slower than C and its largest feasible size within the budget is n = 1024; (3) at power-of-two sizes Java is faster than C, but this is caused by cache conflicts in C's memory layout and not by the language itself: at n ± 1 C is the fastest; (4) C uses about the memory of the data, Java adds the JVM and garbage, and Python needs about 4 times more memory per number. Open questions for the next assignments: changing the loop order (i-k-j) or using blocks should remove the column access problem.

7. References and materials

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[2] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., 2007.

[3] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] A. Georges, D. Buytaert and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in Proc. OOPSLA, 2007, pp. 57-76.

[5] Python Software Foundation, “time: Time access and conversions,” Python 3 documentation. https://docs.python.org/3/library/time.html

[6] Oracle, “Class System,” Java SE 25 API documentation. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/System.html

[7] G. Rodola, “psutil documentation.” https://psutil.readthedocs.io

[8] NumPy Developers, “Random Generator,” NumPy documentation. https://numpy.org/doc/stable/reference/random/generator.html

[9] AMD, “Software Optimization Guide for AMD Family 19h Processors,” pub. 56665, 2020.

Materials. Everything is in https://github.com/Pablo5250/matrix-multiplication, folder assignment1/: source code, config.json, README.md with exact commands (run_all.bat does the whole study), raw results (results/raw/times.csv, runs.csv), summary tables, analysis scripts and figures. The input matrices are not stored; data/generate_inputs.py recreates them and data/manifest.json lets you check that they are identical (SHA-256).

Use of generative AI. I used Claude (Anthropic) as a support tool to help write the code and draft the text of this report. I reviewed all the code and the text, ran every test and measurement on my own computer, and checked the results. I am responsible for the code, the data and the conclusions.